

Adversarial Co-Evolution of RL and LLM Agents in Gin Rummy: A Gold-Standard Study of What Makes a Lightweight Agent Strong

Nima Kelidari Mahdi Salmani Mohammadsaeed Haghi
{kelidari, salmanis, haghim}@usc.edu
University of Southern California

Abstract

Training a reinforcement-learning (RL) agent for an imperfect-information card game runs into the *opponent bottleneck*: a policy is only as good as the opponents it practises against. We study this in Gin Rummy by building a complete framework—an action-masked PPO/TRPO learner, a hand-coded *gold-standard* (provably optimal) opponent used purely as a yardstick, a distributed master-worker stack that lets a 7B language model (LLM) act inside the game, an opponent-curriculum system, and a browser game—then benchmarking nearly every plausible way to strengthen the agent on a single objective: win-rate against the perfect player. Across 100+ controlled experiments we find that (i) the gold standard beats every learned agent (70–99%) yet *gins under 2% of the time*, winning by low-deadwood knocking, and an RL agent rediscovers this even when gin is rewarded $3\times$ more than a knock; (ii) imitation learning (DAgger), dense short-term reward shaping, and live LLM-in-the-loop training all fail to beat plain curriculum RL—each for an identifiable reason; and (iii) the levers that *do* help—a trust-region algorithm (TRPO), a knock-oriented reward, a rising opponent curriculum, warm-starting, and keeping the best checkpoint—stack to push a lightweight agent from the $\sim 30\%$ self-play champion to **34%** against perfect play. We report the successful engineering together with a critical, honestly-negative analysis of why standard hybrid methods do not surpass pure RL here, and argue the remaining gap needs search/planning or full-history memory rather than further reward tuning.

1 Introduction

Reinforcement learning excels at sequential decision-making but is data-hungry and, crucially, *bootstraps off its opponents*: train against a weak player and the policy learns to exploit weakness rather than to play well. Strong opponents are exactly what is scarce—hand-built solvers are game-specific, and humans do not scale. Large language models (LLMs) are an appealing shortcut: they bring broad, common-sense strategy zero-shot. This project asks, in the concrete testbed of Gin Rummy, **how close a small, fast RL agent can get to optimal play, and which training ideas actually move it there.**

Contributions. (1) A reusable framework coupling masked-PPO/TRPO RL, a *provably optimal* benchmark agent, a distributed LLM-serving stack, an opponent curriculum, and a web game (Sec. 2). (2) A gold-standard benchmark and a clean empirical finding: the optimal policy almost never gins, and RL rediscovers this regardless of reward (Sec. 3, 4.6). (3) A comparative study of ~ 10 training regimes on one yardstick—win-rate vs. the perfect player—with the *reason* each succeeds or fails (Sec. 4, Fig. 3), including honest negatives for DAgger, dense rewards, learned embeddings, and live LLM-in-the-loop. (4) A recipe stacking the helpful levers to reach 34% vs. perfect play (Sec. 4.7, 5).

2 The Framework

Environment and learner. We use PettingZoo/RLCard `gin_rummy_v4`; the observation is four 52-card planes (hand, top discard, known picks, rest). Our `MaskedGinRummyPolicy` sets illegal-action logits to $-\infty$ so the agent only ever selects a legal move; agents act by taking the highest-probability *legal* action (no random fallback). **Gold standard.** A hand-coded optimal player computes exact best melds via RLCard’s solver and knocks the moment it should; it is the yardstick and *never* trains the RL agent. **Distributed LLM.** A single rollout fires tens of thousands of opponent queries; at 0.5–3 s per 7B call a naive loop takes hours. A master node load-balances over self-registering GPU workers behind a suit-symmetry cache, speaking the unchanged Ollama API. A key infrastructure result: staging 7B weights on scratch/BeeGFS rather than home NFS cuts worker load from ~ 28 min to ~ 27 s (**62** $\times$), and a 14-worker pool sustains ~ 32 queries/s. **Curriculum + game.** A scheduler ramps opponents (random $\rightarrow$ past selves $\rightarrow$ strong models); a zero-dependency browser client lets humans play any agent. Fig. 1 shows how the pieces connect.

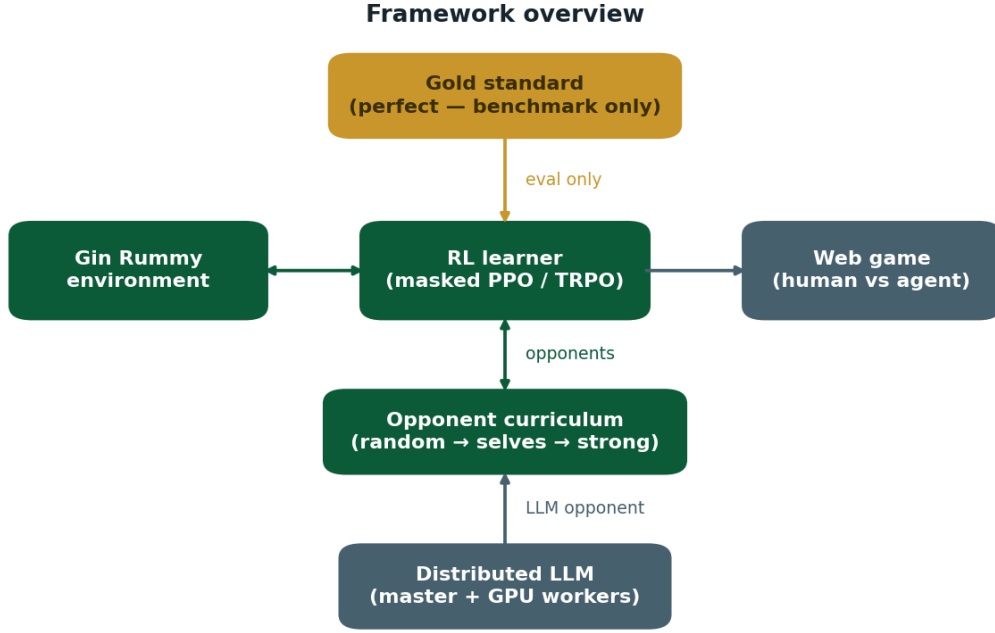


Figure 1: System overview. The learner trains in the environment against a curriculum of opponents (and, in dedicated runs, the LLM); the gold standard is used for *evaluation only*.

3 The Gold-Standard Benchmark and a Surprise

To grade everything on one honest scale we built a provably optimal player and ran it against the whole zoo of trained agents (Fig. 2, 600 games each). It wins 70.2% vs. the self-play champion and 70–99% vs. all others. The surprise is the mechanism: the optimal player **gins only 0.7–1.7% of the time**, despite gin scoring the most points. It wins by *knocking early with low deadwood*. This inverts a natural intuition—chasing gin is a beginner’s trap—and reframes every reward experiment below.

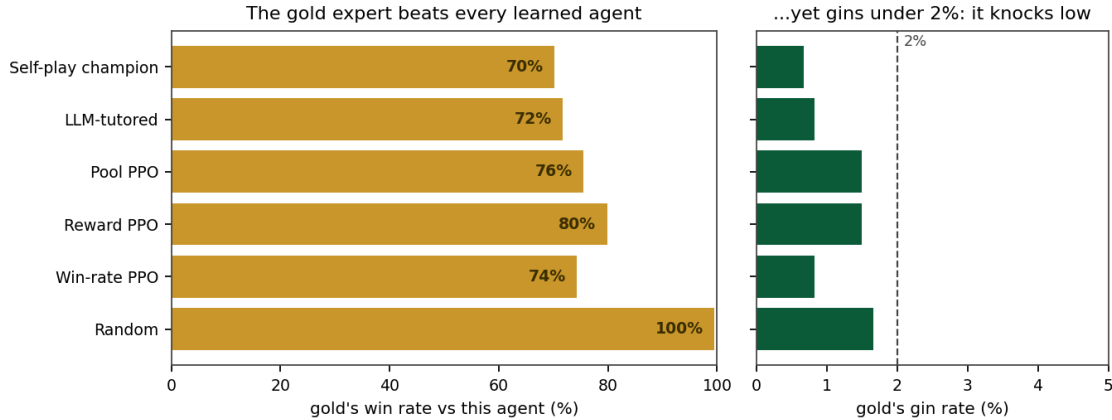


Figure 2: The optimal player beats every learned agent (left) yet gins under 2% of the time (right): it wins by low-deadwood knocking, not by chasing gin.

4 What We Tried: A Comparative Study

We benchmarked nearly every reasonable training regime on the same metric—best win-rate vs. the perfect player (Fig. 3)—and, for each, identify *why* it lands where it does.

4.1 RL backbone and reward shaping

Action-masked PPO saturates against random opponents: an eight-configuration sweep lands every run in a 98.3–99.6% band (indistinguishable at 1000-game eval), and a broader effort of 40+ reward-shaping runs shows the gin/knock reward ratio is a real behavioural knob—a knock-biased reward yields 97% knocks (win 99.4%) while a gin-biased one

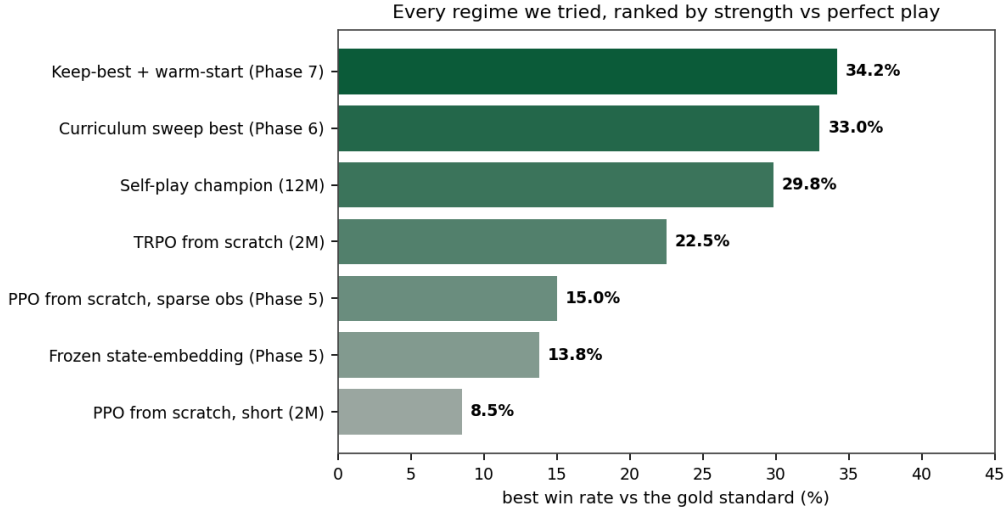


Figure 3: Every regime we benchmarked, ranked by measured win-rate against the perfect player.

yields 22% gins (win 98.3%). *Reason*: random play is too weak to teach strategy, so the easy metric saturates; the reward ratio sets the agent’s risk appetite. This motivates stronger opponents and a principled reward.

4.2 Self-play and curriculum

Self-play is a free rising curriculum: a 3M-step self-play agent beats its own progenitor 61%. But an unguided pool of past selves *diverged* after ~ 10 M steps. *Reason*: self-play creates difficulty for free but can chase non-transitive cycles; the schedule and opponent mix matter, motivating the controlled sweep of Sec. 4.6.

4.3 Why the hybrid methods fail (honest negatives)

Live LLM-in-the-loop. Mid-size LLMs (Qwen2.5-7B, gpt-oss-20b) play real Gin Rummy with a chain-of-thought prompt (100% legal moves) and even beat our self-play agent 3–2 in short matches—but at 9–27s/move they are orders of magnitude too slow for the millions of moves RL needs; even with the distributed stack, online training would take weeks. Vision-LLMs failed outright ($\sim 10\%$ legal moves): it is a text-reasoning task. **Imitation (Dagger).** Cloning an expert move-by-move collapses to near-zero win-rate. *Reason*: the student mimics actions in familiar states but never internalises the *reasoning* (e.g. tracking the opponent), so it fails off-distribution—“mimicking a move” $\neq$ “understanding strategy.” **Dense/short-term rewards.** Rewarding each step (horizon $L = 2, 5$) induces myopia: $L = 2$ shows no learning and $L = 5$ saturates after ~ 500 k steps, “farming” instant points rather than trying to win. *Reason*: dense shaping distracts from the true sparse objective. The redline that follows: we use the LLM/expert only to *inform design*, never to supervise the policy directly.

4.4 Algorithm: PPO vs. TRPO

Under identical masking, curriculum, and reward, TRPO beats PPO at 2M steps on every opponent and both seeds: 50.5% vs. 31.0% champion and 22.5% vs. 15.0% gold (Fig. 4a). *Reason*: TRPO’s trust-region update is more robust to the sparse-terminal, non-stationary self-play target than PPO’s fixed clip. (GRPO/DPO are LLM-alignment objectives with no per-move environment analogue and do not apply.)

4.5 State representation

We compressed the large sparse observation into a dense 20-d embedding two ways—self-supervised temporal similarity (states near in a game are similar) and an LLM judging strategic similarity—and used each (frozen) as the policy input. Both *lose* to the raw sparse input vs. gold (6–14% vs. 15%). *Reason*: a frozen bottleneck good at “are these states similar” still discards detail the policy needs; size and un-freezing did not recover it.

4.6 The curriculum sweep and the key finding

A 30-run one-factor sweep over algorithm $\times$ reward $\times$ curriculum lands every recipe near champion strength (~ 30 –33% vs. gold), but yields one clean, sturdy result (Fig. 4b): **the gin-rate stays under 1% regardless of reward**—even paying $3\times$ for a gin does not raise it above the balanced baseline. *Reason*: like the optimal player, the policy discovers that chasing gin loses; you cannot bribe it into the bad habit. The reward that actually helps does the opposite—a small “knock earlier” penalty that yields the shortest games and the best play. Fig. 5 shows the mechanism: win-rate against the champion climbs as the curriculum swaps in tougher opponents (random $\rightarrow$ pool $\rightarrow$ self $\rightarrow$ strong), and the late dip on one run is exactly the drift that motivates keep-the-best (Sec. 4.7).

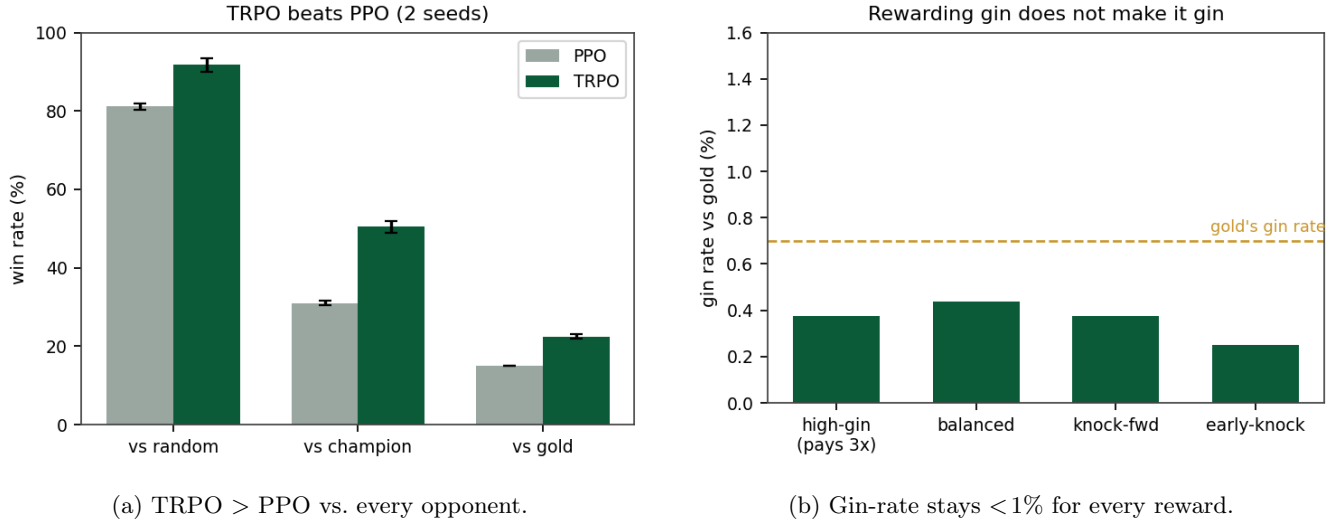


Figure 4: Algorithm and reward studies (2 seeds each).

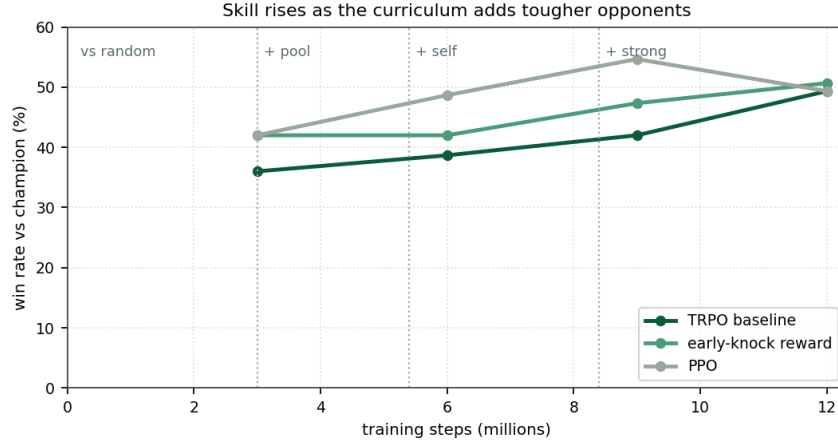


Figure 5: Skill (win-rate vs. the champion) rises through the curriculum stages; the late dip motivates best-checkpoint selection.

4.7 Keep-the-best and warm-start (the recipe)

A final series stacks the helpful levers: *warm-start* from the champion, the knock-oriented reward plus a dense *deadwood-reduction* signal (a policy-invariant nudge toward the optimal style, computed from the agent’s own public hand), a curriculum seeded with the strongest prior agents, and—critically—*keeping the best checkpoint* (training drifts past its peak, so we evaluate every 1M steps and ship the best). Re-evaluated at 2000 games, the strongest agent reaches $34.2\% \pm 2.1\%$ vs. gold and 51.4% vs. the old champion—our best, up from the champion’s $\sim 30\%$.

5 What Mattered, and the Bottom Line

Stacking the helpful levers took a lightweight agent from the champion’s $\sim 30\%$ to 34% against *perfect* play (Fig. 6) and produced one reusable scientific result (optimal play rarely gins). The honest ceiling is the mid-30s percent: reward, algorithm, and opponent tuning plateau there.

6 Conclusion and Future Work

We built the framework, the benchmark, and the comparative study to answer—fairly—how strong a lightweight RL agent can be in Gin Rummy and what makes it so. Pure curriculum RL, sharpened with a trust-region algorithm, a knock-oriented reward, warm-starting, and best-checkpoint selection, is the winner; the popular hybrid shortcuts (imitation, dense reward, live LLM-in-the-loop) each fail for a concrete, diagnosed reason. Surpassing the mid-30s ceiling against optimal play almost certainly requires a *different class* of method—search/planning such as counterfactual-regret minimization (which powers superhuman poker), or an agent with explicit full-history memory—rather than further reward or curriculum tuning. The framework, the gold-standard benchmark, and the distributed LLM dataset pipeline are the reusable substrate for that next step (e.g. offline RL on a large LLM-vs-LLM corpus). All figures regenerate

Idea	Verdict	Why
Keep the best checkpoint	helps	training drifts past its peak; saving the best recovers 2–3 pts
Warm-start from the champion	helps	start strong, then specialise
TRPO over PPO	helps	safer policy steps suit sparse, shifting self-play
Reward knocking, not gin	helps	copies the optimal low-risk style
Rising opponent curriculum	helps	always a fair-but-harder challenge
Pay more for gin	no effect	the agent refuses the bad habit at any bribe
Learned state embeddings	fails	a frozen bottleneck discards useful detail
Imitation (DAGger)	fails	copies moves, not the reasoning behind them
Dense short-term rewards	fails	myopia—greedy for points, blind to winning
Live LLM-in-the-loop	fails	strong but far too slow for millions of moves

Table 1: Which ideas moved the needle against the perfect player, across 100+ runs.

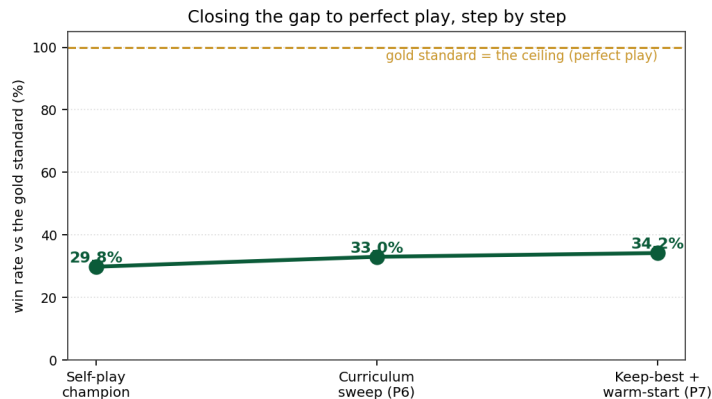


Figure 6: Closing the gap to perfect play, step by step.

from saved JSON; live training curves are on Weights & Biases.

References

- [1] J. Schulman et al. Proximal Policy Optimization Algorithms. *arXiv:1707.06347*, 2017.
- [2] J. Schulman et al. Trust Region Policy Optimization. *ICML*, 2015.
- [3] S. Huang and S. Ontañón. A Closer Look at Invalid Action Masking in Policy Gradient Algorithms. *arXiv:2006.14171*, 2022.
- [4] S. Ross, G. Gordon, D. Bagnell. A Reduction of Imitation Learning to No-Regret Online Learning (DAGger). *AISTATS*, 2011.
- [5] N. Brown and T. Sandholm. Superhuman AI for multiplayer poker. *Science* 365(6456):885–890, 2019.
- [6] J. Terry et al. PettingZoo: Gym for Multi-Agent Reinforcement Learning. *NeurIPS*, 2021.
- [7] D. Zha et al. RLCARD: A Toolkit for Reinforcement Learning in Card Games. *arXiv:1910.04376*, 2019.
- [8] A. Raffin et al. Stable-Baselines3. *JMLR* 22(268):1–8, 2021.
- [9] Qwen Team. Qwen2.5 Technical Report. *arXiv:2412.15115*, 2024.